EXPERIMENT-08

Deploy a fine-tuned language model on Hugging Face

AIM: Deploy a fine-tuned language model on Hugging Face Spaces or Streamlit for customer support, and evaluate it using ROUGE and BLEU metrics.

Dataset: Customer service dataset or Bank FAQ dataset.

CODE:

```
# ============================================================
# CUSTOMER SUPPORT FAQ MODEL (mT5-small) - FULL COLAB CODE
# ============================================================

# ----------------------------
# STEP 21: INSTALL DEPENDENCIES
# ----------------------------
!pip install -q transformers datasets evaluate sentencepiece sacrebleu
rouge-score

# ----------------------------
# STEP 21: IMPORT LIBRARIES
# ----------------------------
import torch
import pandas as pd
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
from datasets import Dataset
from evaluate import load
from sklearn.model_selection import train_test_split
from google.colab import files

# ----------------------------
# STEP 21: DEVICE SETUP
# ----------------------------
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

# ----------------------------
# STEP 21: UPLOAD BANK FAQ CSV
# ----------------------------
uploaded = files.upload()
csv_file = list(uploaded.keys())[0]
print("Uploaded file:", csv_file)

df = pd.read_csv(csv_file)
print("\nDataset Preview:")
print(df.head())
print("\nColumns:", df.columns)

# ----------------------------
```

```
# STEP 21: TRAIN-TEST SPLIT
# ----------------------------
train_q, test_q, train_a, test_a = train_test_split(
    df["question"].tolist(),
    df["answer"].tolist(),
    test_size=0.3,
    random_state=42
)

train_dataset = Dataset.from_dict({
    "question": train_q,
    "answer": train_a
})

test_dataset = Dataset.from_dict({
    "question": test_q,
    "answer": test_a
})

# ----------------------------
# STEP 22: LOAD MODEL & TOKENIZER
# ----------------------------
model_name = "google/mt5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name).to(device)

# ----------------------------
# STEP 22: TOKENIZATION
# ----------------------------
max_len = 128

def preprocess(batch):
    inputs = tokenizer(
        batch["question"],
        padding="max_length",
        truncation=True,
        max_length=max_len
    )

    targets = tokenizer(
        batch["answer"],
        padding="max_length",
        truncation=True,
        max_length=max_len
    )

    labels = [
        [(t if t != tokenizer.pad_token_id else -100) for t in seq]
        for seq in targets["input_ids"]
    ]
```

```
    inputs["labels"] = labels
    return inputs

train_tokenized = train_dataset.map(preprocess, batched=True)
test_tokenized = test_dataset.map(preprocess, batched=True)

train_tokenized.set_format("torch")
test_tokenized.set_format("torch")

# ----------------------------
# STEP 22: TRAINING (3 EPOCHS)
# ----------------------------
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-5)

model.train()
print("\n=== TRAINING STARTED ===")

for epoch in range(3):
    total_loss = 0
    for batch in train_tokenized:
        batch = {k: v.unsqueeze(0).to(device) for k, v in
batch.items()}
        optimizer.zero_grad()
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(f"Epoch {epoch+1}/3 | Avg Loss:
{total_loss/len(train_tokenized):.4f}")

# ----------------------------
# STEP 23: EVALUATION
# ----------------------------
model.eval()
rouge = load("rouge")
bleu = load("bleu")

predictions = []
references = []

with torch.no_grad():
    for batch in test_tokenized:
        input_ids = batch["input_ids"].unsqueeze(0).to(device)

        outputs = model.generate(
            input_ids,
            max_new_tokens=64,
            num_beams=4,
            do_sample=False
```

```
        )

        pred = tokenizer.decode(outputs[0], skip_special_tokens=True)

        ref = tokenizer.decode(
            [x for x in batch["labels"].tolist() if x != -100],
            skip_special_tokens=True
        )

        predictions.append(pred)
        references.append(ref)

# ----------------------------
# STEP 23: METRIC COMPUTATION
# ----------------------------
rouge_results = rouge.compute(
    predictions=predictions,
    references=references
)

bleu_results = bleu.compute(
    predictions=[p.split() for p in predictions],
    references=[[r.split()] for r in references]
)

print("\n=== EVALUATION RESULTS ===")
print(f"ROUGE-L Score: {rouge_results['rougeL']:.4f}")
print(f"BLEU-4 Score : {bleu_results['bleu']:.4f}")

# ----------------------------
# STEP 23: SAMPLE OUTPUTS
# ----------------------------
print("\n=== SAMPLE PREDICTIONS ===")
for i in range(min(3, len(predictions))):
    print("Q   :", test_q[i])
    print("Pred:", predictions[i])
    print("Ref :", test_a[i])
    print("-" * 60)
```

OUTPUT:

```
=== TRAINING ===
Epoch 1/3 | Avg Loss: 13.6484
Epoch 2/3 | Avg Loss: 8.0355
Epoch 3/3 | Avg Loss: 6.8788
```

ROUGE-L Score: 0,.0145
BLEU 4 Soore 0.0002

SAMPLE OUTPUT:

Q : Can I sell/redeem schemes bought offline
through online Pre<l: <extraid0>.
Ref : No, only those schemesbought through the online oode can be sold /redee1edthrough this service,

Q : In me of existing investors, when a n d h o w ill the KYC
norms be introduced Pre<l: <extraid0>.
Ref : KYC normsare applicable to all investors, It is in the interest of all investors to obtain KYCAcknowledgement and submit it to the Mutual Fund to avoidany inconvenience in thefuture,

Q : What is an injuryor
injuries Pre<l: <extraid0>.
Ref : Injury or injuries means any physical, external or accidental bodily injuryoccurring suddenly in timeand resulting solely and independentlyof anyother cause or any physical defect or infirmity existing before